

Divide & Conquer Algorithms:

Structure of divide-and-conquer algorithms, examples: Binary search, quick sort, Merge sort, Strassen

Multiplication; Max-Min problem

❖ Divide & Conquer (DAC):- for ex- as- $f(6)$

❑ Divide the big problem into some subproblems. as- $6*f(5)$

❑ Conquer the subproblem by calling recursively until we will get subproblem solution. as- $f(5)=120$

❑ Combine the subproblem solution so that we will get the final problem solution. as- $6*120=720$

❖ DAC Abstract Alg

▪ $DAC(a, p, q) \rightarrow T(n)$

▪ {

▪ if (small(a, p, q)) $\rightarrow O(1)$

▪ return(solution(a, p, q)) $\rightarrow O(1)$

▪ else {

▪ 1. $m = \text{Divide}(a, p, q) \rightarrow f_1(n)$

▪ 2. $b_1 = DAC(a, p, m) \rightarrow T(n/2)$

▪ $b_2 = DAC(a, m+1, q) \rightarrow T(n/2)$

▪ 3. $C = \text{Combine}(b_1, b_2) \rightarrow f_2(n)$

▪ return(C)

▪ }

▪ }

DAC – Algo – T.C – Finding

$T(n) = \{O(1),$ if n is small

$f_1(n) + T(n/2) + T(n/2) + f_2(n),$ if n is big

$$\Rightarrow T(n) = 2T(n/2) + f(n)$$

- $n/2$ - size of 1 sub problem
- 2 - it indicates 2 subproblems
- $T(n/2)$ - means T.C of 1-subproblem
- $f(n)$ – means amount of time required for both divide & combine

- ☐ In general:-
- ☐ $T(n) = aT(n/b) + f(n)$
- ☐ Where,
- ☐ $a \geq 1, b > 1$
- ☐ $f(n)$ are functions
- ☐ so we can say,
- ☐ M.T is actually meant for DAC.

❖ **Applications of DAC:-**

- ❖ Finding maximum & minimum
- ❖ Power of an element
- ❖ Binary Search
- ❖ Merge Sort
- ❖ Quick Sort
- ❖ Selection Procedure
- ❖ Strassen's Matrix Multiplication
- ❖ Counting Inversions
- ❖ Finding max-contiguous subarray sum

Divide & Conquer Algorithms:-

Structure of divide-and-conquer algorithms,
examples:

- Binary search,
- quick sort,
- Merge sort,
- Strassen Multiplication; Max-Min problem

1. Divide:

- Break down the original problem into smaller subproblems.
- Each subproblem should represent a part of the overall problem.
- The goal is to divide the problem until no further division is possible.

2. Conquer:

- Solve each of the smaller subproblems individually.
- If a subproblem is small enough (often referred to as the “base case”), we solve it directly without further recursion.
- The goal is to find solutions for these subproblems independently.

Binary Search Algorithm

0	1	2	3	4	5	6	7	8	9
-5	-2	0	1	2	4	5	6	7	10
Low		Middle				High			
-5	-2	0	1	2	4	5	6	7	10
					Low	Middle		High	
-5	-2	0	1	2	4	5	6	7	10
								Low	High
								Middle	

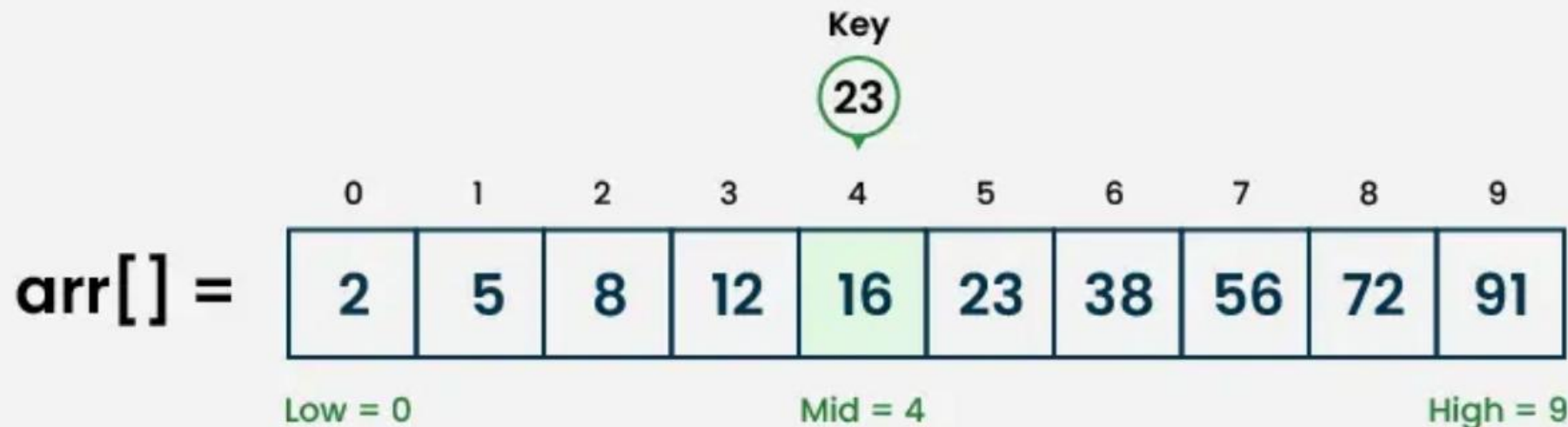
Initially

Find Key = 23 using Binary Search

	0	1	2	3	4	5	6	7	8	9
arr[] =	2	5	8	12	16	23	38	56	72	91

Step 1

Key (i.e., 23) is greater than current mid element (i.e., 16). The search space moves to the right.



Step 2

Key is less than the current mid 56.
The search space moves to the left.

arr[] =

0	1	2	3	4	5	6	7	8	9
2	5	8	12	16	23	38	56	72	91
					Low = 5		Mid = 7	High = 9	

Step 3

If the key matches the value of the mid element, the element is found and stop search.



The **Binary Search Algorithm** can be implemented in the following two ways

- Iterative Binary Search Algorithm
- Recursive Binary Search Algorithm

Iterative Binary Search Algorithm

Algorithm Steps:

1. Initialize two pointers, `low` and `high`, representing the start and end indices of the array.
2. While `low` is less than or equal to `high`:
 - Calculate the middle index: `mid = (low + high) / 2`.
 - If the element at `mid` equals the target, return the index.
 - If the target is greater than the middle element, narrow the search to the right half by setting `low = mid + 1`.
 - If the target is smaller, narrow the search to the left half by setting `high = mid - 1`.

Pseudocode: Iterative

```
function binarySearchIterative(arr, target):
```

```
    low  $\leftarrow$  0
```

```
    high  $\leftarrow$  length(arr) - 1
```

```
    while low  $\leq$  high:
```

```
        mid  $\leftarrow$  (low + high) / 2
```

```
        if arr[mid] = target:
```

```
            return mid
```

```
        else if arr[mid] < target:
```

```
            low  $\leftarrow$  mid + 1
```

```
        else:
```

```
            high  $\leftarrow$  mid - 1
```

```
    return -1
```

Recursive Binary Search Algorithm

Algorithm Steps:

1. Check if the current search bounds (`low` to `high`) are valid.
2. Compute the middle index: `mid = (low + high) / 2`.
3. If the element at `mid` matches the target, return the index.
4. If the target is greater than the element at `mid`, recursively search the right subarray.
5. If the target is less, recursively search the left subarray.
6. If the search bounds become invalid (`low > high`), return `-1` to indicate the target is not present.

Pseudocode: Recursive

```
function binarySearchRecursive(arr, low, high, target):  
    if low > high:  
        return -1  
  
    mid  $\leftarrow$  (low + high) / 2  
  
    if arr[mid] = target:  
        return mid  
    else if arr[mid] < target:  
        return binarySearchRecursive(arr, mid + 1, high, target)  
    else:  
        return binarySearchRecursive(arr, low, mid - 1, target)
```


Why is $\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$ preferred over $\text{mid} = (\text{low} + \text{high}) / 2$?

- A) It's faster
- B) It prevents overflow in large integers
- C) It uses less memory
- D) It sorts the array

✓ So why not always use $(low + high) / 2$?

Because if `low` and `high` are very large, say:

cpp

```
int low = 2'000'000'000;  
int high = 2'100'000'000;
```

- `low + high = 4'100'000'000` → overflows a 32-bit int (max $\approx$ 2.1 billion)
- The result becomes **negative or incorrect**

But:

cpp

```
int mid = low + (high - low) / 2;
```

```
// (high - low) = 100,000,000 → safe!
```

```
// mid = 2,000,000,000 + 50,000,000 = 2,050,000,000 →  correct
```

Expression	Result (Small Numbers)	Result (Large Numbers)	Risk of Overflow	Recommendation 
<code>(low + high) / 2</code>	 Correct (e.g., 5)	 Incorrect or crashes	 Yes	 Avoid for safety
<code>low + (high - low) / 2</code>	 Correct (e.g., 5)	 Correct (e.g., 2,050,000,000)	 No	 Preferred and safe

What does the expression `sizeof(arr) / sizeof(arr[0])` compute?

- A) The sum of all array elements
- B) The size of one element in the array
- C) The number of elements in the array
- D) The memory address of the array

What is the time complexity of `int n = sizeof(arr) / sizeof(arr[0]);`?

- A) $O(n)$
- B) $O(\log n)$
- C) $O(1)$
- D) Depends on array size

```
int arr[] = {10, 20, 30, 40, 50};
```

Assume:

- Each `int` = 4 bytes
- Total elements = 5

```
sizeof(arr)      = 5 × 4 = 20 bytes
```

```
sizeof(arr[0])   = 4 bytes
```

```
n = sizeof(arr) / sizeof(arr[0])
```

```
   = 20 / 4
```

```
   = 5
```

Step 1: Write the Recurrence Relation

Binary Search works by dividing the array in half each time, so the recurrence relation is:

$$T(n) = T(n/2) + c$$

- You reduce the problem size by half: $n \rightarrow n/2$
- You do **constant** work c at each step (comparing middle element)

Step 2: Match the Recurrence to Master Theorem Format

The **Master Theorem** solves recurrences of the form:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

Where:

- a = number of subproblems
- b = factor by which the problem size is divided
- $f(n)$ = the work done outside the recursive calls

From binary search:

- $a = 1$ (1 subproblem)
- $b = 2$ (size is divided by 2)
- $f(n) = c = O(1)$

So:

$$T(n) = 1 \cdot T(n/2) + O(1)$$



Step 3: Apply the Master Theorem Cases

Now check which case of the Master Theorem this fits.

Compare $f(n) = O(1)$ with $n^{\log_b a}$:

- $\log_b a = \log_2 1 = 0$
- So $n^{\log_b a} = n^0 = 1$
- $f(n) = O(1) = \Theta(n^0)$

This matches **Case 2** of the Master Theorem:

Case 2: If $f(n) = \Theta(n^{\log_b a})$, then:

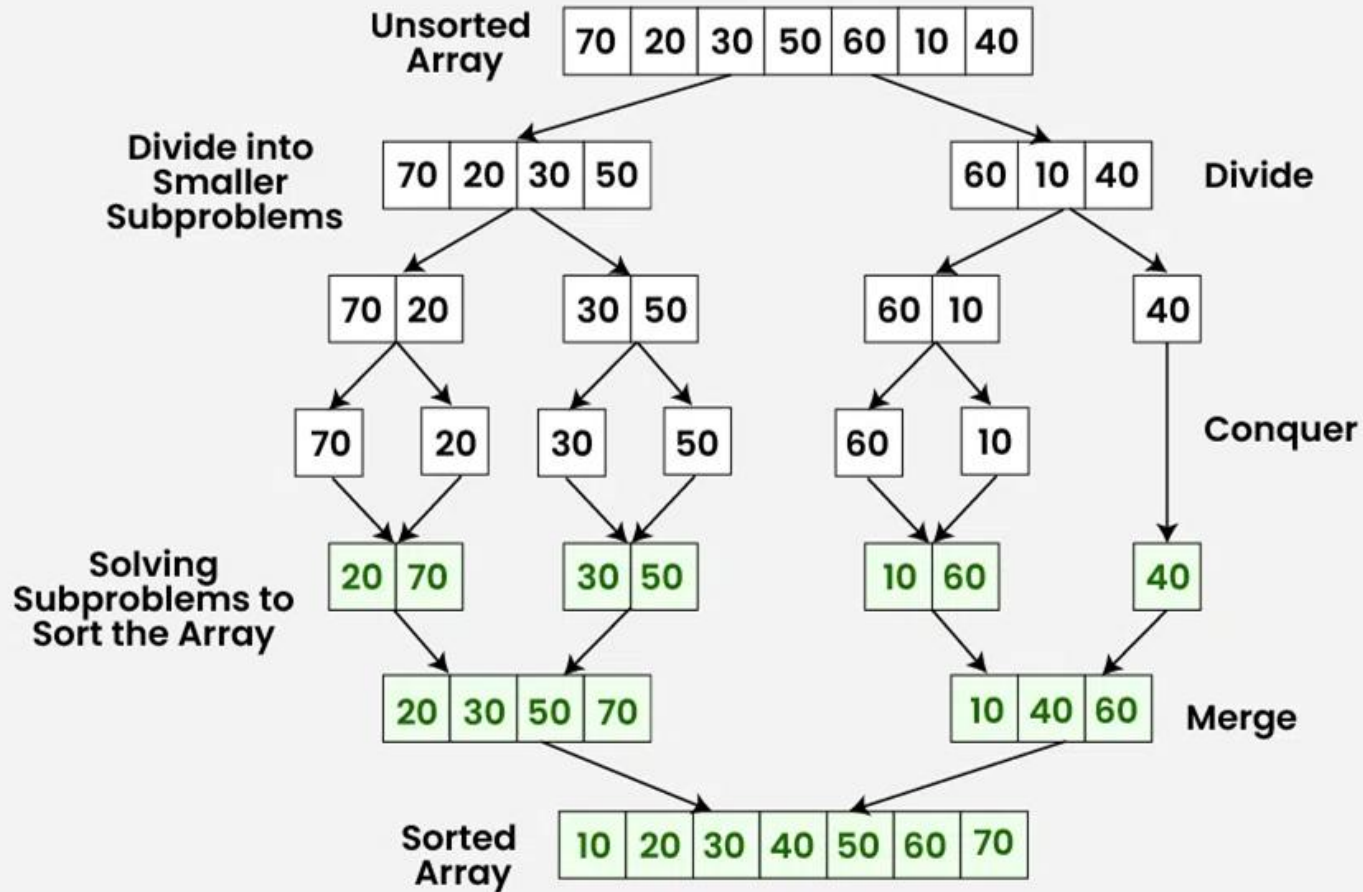
$$T(n) = \Theta(n^{\log_b a} \cdot \log n) = \Theta(\log n)$$



Time & Space Complexity

Version	Time Complexity	Space Complexity
Iterative	$O(\log n)$	$O(1)$
Recursive	$O(\log n)$	$O(\log n)$ (due to recursion stack)

Working of Divide & Conquer Algorithm



Merge Sort - Solve this question

[48, 16, 53, 4, 10, 83, 14]

0	1	2	3	4	5	6
48	16	53	4	10	83	14

$$\frac{0+6}{2} = 3$$

0	1	2	3
48	16	53	4

$$\frac{0+3}{2} = 1$$

4	5	6
10	83	14

0	1
48	16

2	3
53	4

48

16

53

4



0	1	2	3	4	5	6
4	16	48	53	10	14	83

INPUT



```
void merge(int arr[], int leftIndex, int middleIndex, int rightIndex)
```

```
{
```

```
    int i, j, k;
```

```
    int n1 = middleIndex - leftIndex + 1;
```

n1=4

```
    int n2 = rightIndex - middleIndex;
```

n2=3

```
    int L[n1], R[n2];
```

Step 1:

0	1	2	3	4	5	6
4	16	48	53	10	14	83

INPUT

	0	1	2	3
L[i]	4	16	48	53

0	1	2

R[j]

```
for (i = 0; i < n1; i++)  
{  
    L[i] = arr[leftIndex + i];  
}  
for (j = 0; j < n2; j++)  
{  
    R[j] = arr[middleIndex + 1 + j];  
}
```

$$4 \leq 10$$

	0	1	2	3		0	1	2	
L[i]	4	16	48	53		10	14	83	R[j]

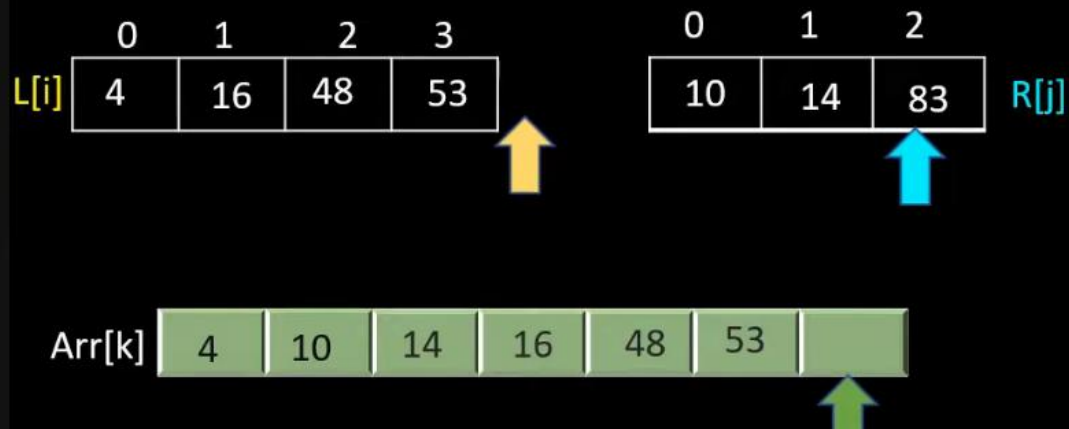
Arr[k]	4						
--------	---	--	--	--	--	--	--

$48 \leq 83$

	0	1	2	3
L[i]	4	16	48	53

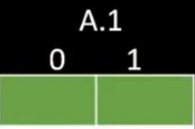
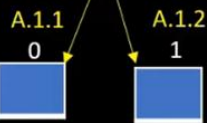
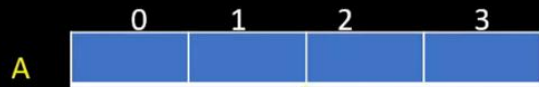
	0	1	2
R[j]	10	14	83

Arr[k]	4	10	14	16	48		
--------	---	----	----	----	----	--	--



```
while (i < n1)
{
    arr[k] = L[i];
    i++;
    k++;
}

2 < 3
while (j < n2)
{
    arr[k] = R[j];
    j++;
    k++;
}
}
```



```

Leftindex=0
rightindex=1
Middleindex=0
void merge(...)
{
.....
}

```

```

void mergeSort(int arr[], int leftIndex, int rightIndex)

```

```

if (leftIndex < rightIndex) {
    int middleIndex=(rightIndex +leftIndex) / 2;
    mergeSort(arr, leftIndex, middleIndex);    // Sort first halves
    mergeSort(arr, middleIndex + 1, rightIndex); // Sort second halves
    merge(arr, leftIndex, middleIndex, rightIndex);
}

```

MAIN

```

if (leftIndex < rightIndex) 1<1
CONDITION FAILS
A.1.2
if (leftIndex < rightIndex) 0<0
CONDITION FAILS
A.1.1
if (leftIndex < rightIndex) 0<1
middleindex=0
Mergesort-1 (0-0)
Mergesort-2 (1-1)
merge
A.1
if (leftIndex < rightIndex) 0<3
Middleindex 1
Mergesort-1 (0-1)
Mergesort-2
merge
A

```

function MergeSort(A, left, right)

if left < right then

mid $\leftarrow$ (left + right) / 2

// Recursively sort first and second halves

MergeSort(A, left, mid)

MergeSort(A, mid + 1, right)

// Merge the sorted halves

Merge(A, left, mid, right)

Step 1: Copying

function Merge(A, left, mid, right):

$n1 \leftarrow \text{mid} - \text{left} + 1$

$n2 \leftarrow \text{right} - \text{mid}$

$L[0..n1 - 1], R[0..n2 - 1]$

 for $i \leftarrow 0$ to $n1 - 1$:

$L[i] \leftarrow A[\text{left} + i]$

 for $j \leftarrow 0$ to $n2 - 1$:

$R[j] \leftarrow A[\text{mid} + 1 + j]$

$i \leftarrow 0$

$j \leftarrow 0$

$k \leftarrow \text{left}$

while $i < n_1$ and $j < n_2$:

if $L[i] \leq R[j]$:

$A[k] \leftarrow L[i]$

$i \leftarrow i + 1$

else:

$A[k] \leftarrow R[j]$

$j \leftarrow j + 1$

$k \leftarrow k + 1$

while $i < n_1$:

$A[k] \leftarrow L[i]$

$i \leftarrow i + 1$

$k \leftarrow k + 1$

while $j < n_2$:

$A[k] \leftarrow R[j]$

$j \leftarrow j + 1$

$k \leftarrow k + 1$

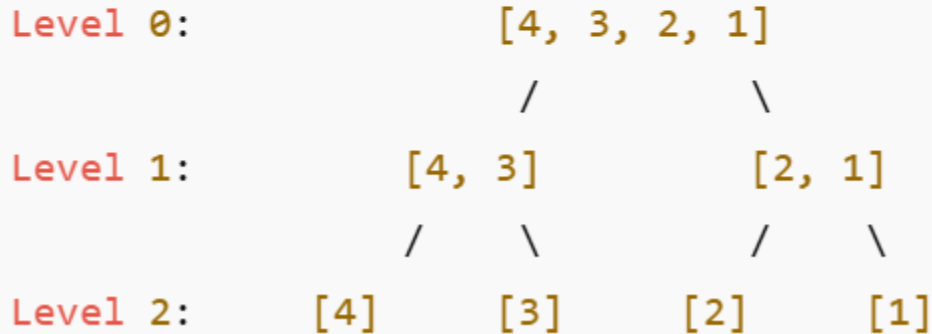
Function	Description	Time Complexity
MergeSort()	Recursively divides the array	$O(\log n)$
Merge()	Merges two sorted subarrays of size n	$O(n)$
Total		 $O(n \log n)$

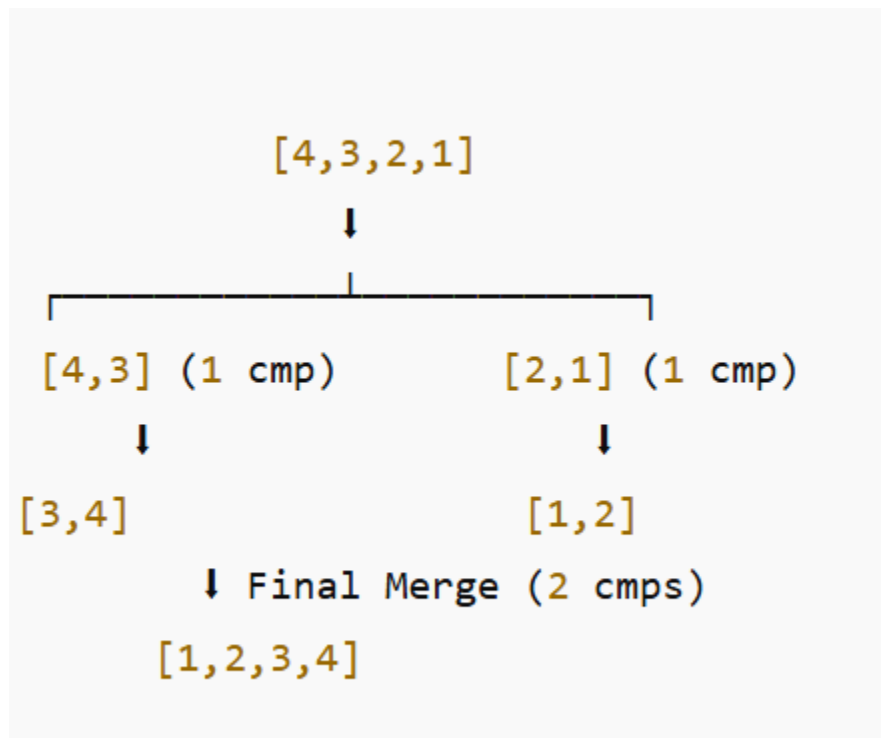
How many comparisons does
Merge Sort make for 4
elements [4,3,2,1]?

How many comparisons does Merge Sort make for 4 elements [4,3,2,1]?

Step 1: Divide Phase (Recursive Splitting)

less





Merge Step	Comparison
[4] + [3]	1
[2] + [1]	1
[3,4] + [1,2]	2
Total	4

How many comparisons does
Quicksort make for 6 elements
[15,4,28,12,18,20]?



Introduction to Quick Sort

- Quick Sort is a **divide-and-conquer** sorting algorithm.
- It works by choosing a **pivot** element and partitioning the array.
- Elements less than the pivot go to the left, greater go to the right.
- The process is **recursively applied** to subarrays.



Advantages of Quick Sort

- 🚀 **Faster in practice** than Merge Sort and others for most inputs.
- 💾 **In-place sorting** (no extra memory needed like Merge Sort).
- 📈 **Average-case time complexity** is efficient: $O(n \log n)$.
- 🔁 **Tail recursion optimization** possible in many implementations.

0	1	2	3	4	5
15	4	28	12	18	20

Pivot=15

↑
isSmaller
slide

↑
isGreater
slide

$A[\text{isSmallerSlide}] \leq \text{pivot}$ $\text{isSmallerSlide}++$		$A[0] \leq 15$ $15 \leq 15$ $\text{isSmallerSlide}++$
---	--	---

0	1	2	3	4	5
15	4	28	12	18	20

Pivot=15

↑ isSmaller slide

↑ isGreater slide

```

A[isSmallerSlide] <= pivot | A[0] <= 15 | A[1] <= 15 | A[2] <= 15
isSmallerSlide++         | 15 <= 15 | 4 <= 15 | 28 <= 15
                           | isSmallerSlide++ | isSmallerSlide++ | Condition fails

```

0	1	2	3	4	5
15	4	28	12	18	20

Pivot=15

↑
isSmaller
slide

↑
isGreater
slide

A[isSmaller] ≤ pivot isSmaller++	A[0] ≤ 15 15 ≤ 15	A[1] ≤ 15 4 ≤ 15	A[2] ≤ 15 28 ≤ 15
	isSmaller++	isSmaller++	Condition fails

A[isGreater] > pivot	20 > 15
	isGreater--

0	1	2	3	4	5
15	4	28	12	18	20

Pivot=15

↑
isSmaller
slide

↑
isGreater
slide

A[isSmaller] ≤ pivot isSmaller++	A[0] ≤ 15 15 ≤ 15	A[1] ≤ 15 4 ≤ 15	A[2] ≤ 15 28 ≤ 15
	isSmaller++	isSmaller++	Condition fails

A[isGreater] > pivot isGreater--	20 > 15	18 > 15	12 > 15
	isGreater--	isGreater--	fails

0	1	2	3	4	5
15	4	12	28	18	20

Pivot=15

↑
isSmaller
slide

↑
isGreater
slide

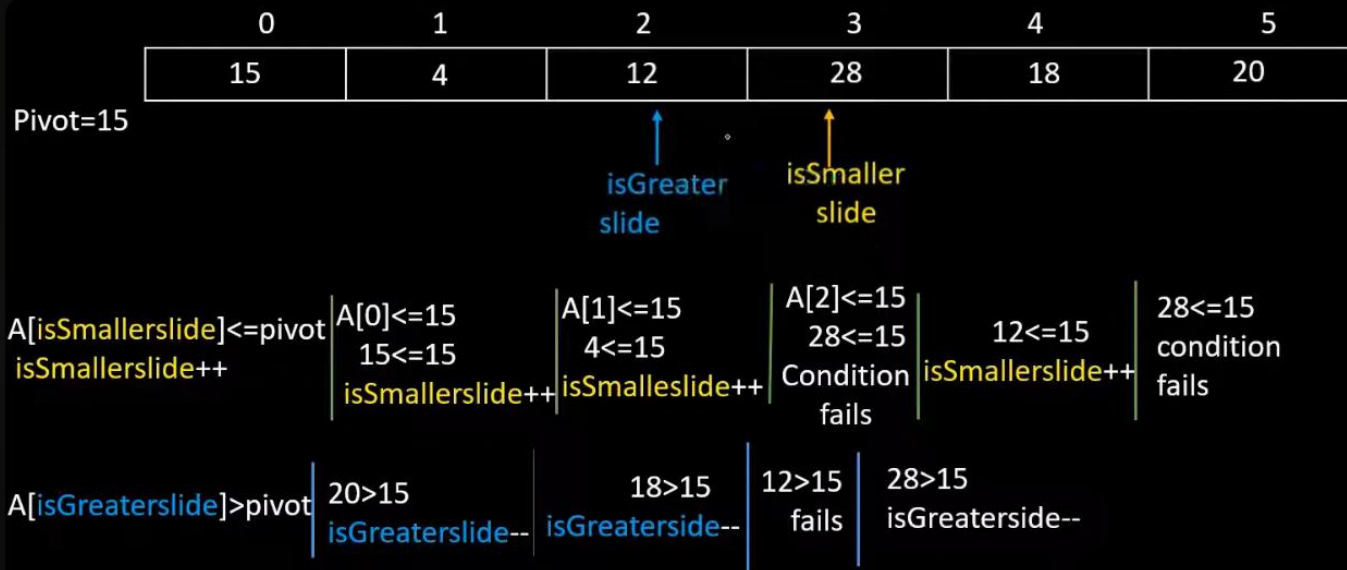
Swap Done

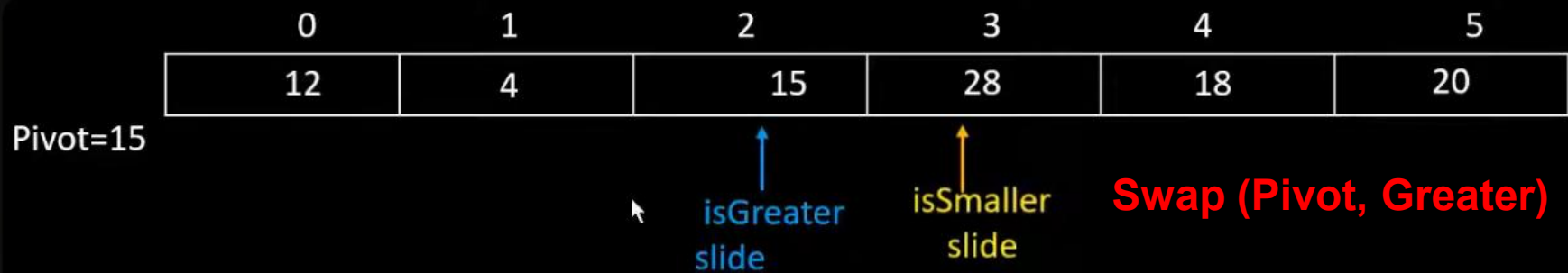
$A[\text{isSmallerSlide}] \leq \text{pivot}$
 $\text{isSmallerSlide}++$

$A[0] \leq 15$ $15 \leq 15$ $\text{isSmallerSlide}++$	$A[1] \leq 15$ $4 \leq 15$ $\text{isSmallerSlide}++$	$A[2] \leq 15$ $28 \leq 15$ Condition fails	$12 \leq 15$ $\text{isSmallerSlide}++$
---	--	---	---

$A[\text{isGreaterSlide}] > \text{pivot}$
 $\text{isGreaterSlide}--$

$20 > 15$ $\text{isGreaterSlide}--$	$18 > 15$ $\text{isGreaterSlide}--$	$12 > 15$ fails
--	--	--------------------







Pivot=12

↑
isSmaller
slide

↑
isGreater
slide

Partition 1

Partition 2

0	1	2	3	4	5
12	4	15	28	18	20

0	1
12	4

Pivot=12

isGreater
isSmaller
slide

Partition 1



0

1

2

3

4

5

12

4

15

28

18

20

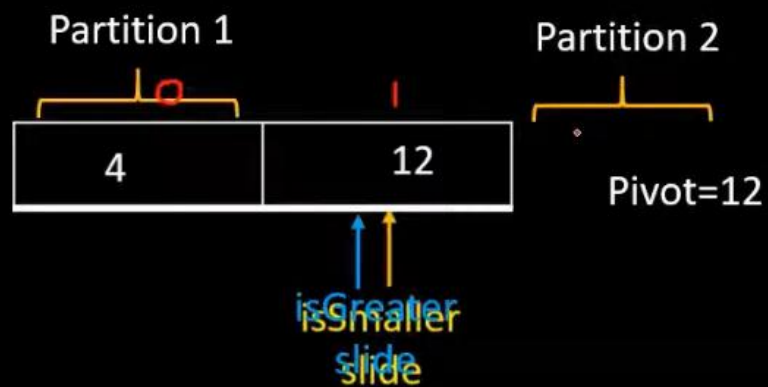
Partition 2

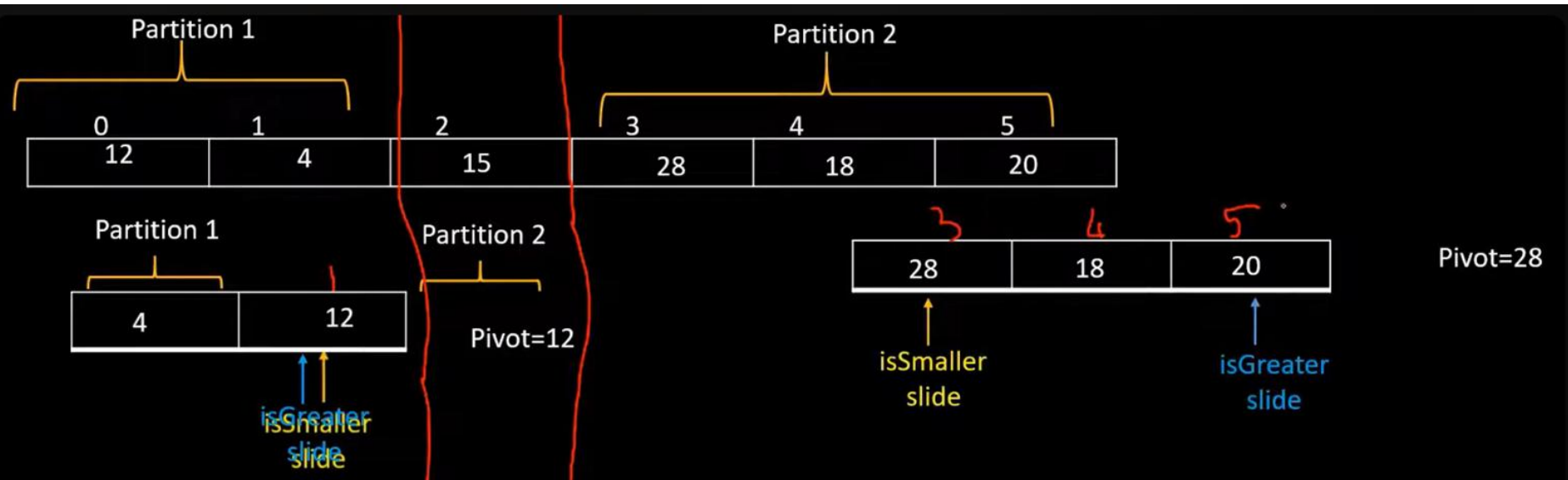


4	12
---	----

Pivot=12

isGreater
isSmaller
slide





Partition 1

Partition 2

0	1	2	3	4	5
12	4	15	28	18	20

Partition 1

Partition 2

4	12
---	----

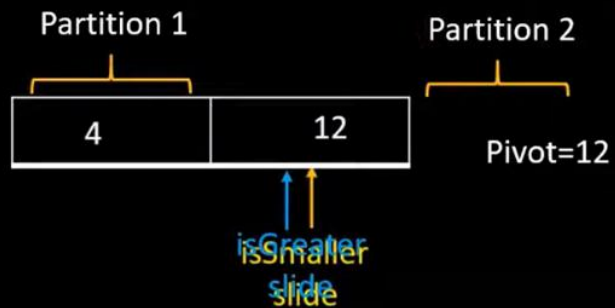
Pivot=12

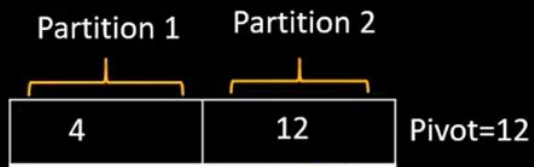
↑ is Greater
↑ is Smaller
slide

3	4	5
28	18	20

Pivot=28

↑ is Greater
↑ is Smaller
slide





↑ isGreater
↑ isSmaller
side slide



↑ isSmaller
side

↑ isGreater
side

Pivot=20



↑ isGreater
↑ isSmaller
side slide



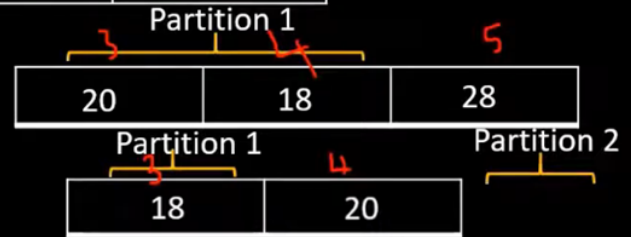
↑ isGreater
↑ isSmaller
side slide

Pivot=20



isGreater
side slide

isSmaller
side slide



isGreater
side slide

isSmaller
side slide

4

18

4	12	15	18	20	28
---	----	----	----	----	----

QuickSort(arr, low, high):

if low < high:

Step 1: Partition the array around a pivot

pivotIndex = Partition(arr, low, high)

Step 2: Recursively sort elements before and after partition

QuickSort(arr, low, pivotIndex - 1)

QuickSort(arr, pivotIndex + 1, high)

Algorithm

Partition(arr, low, high):

pivot = arr[high] # Choose last element as pivot

i = low - 1 # Index of smaller element

for j = low to high - 1:

 if arr[j] <= pivot:

 i = i + 1

swap arr[i] and arr[j]

swap arr[i + 1] and arr[high] # Place pivot in correct position

return i + 1

Algorithm



Time & Space Complexity Comparison

Algorithm	Best Case	Average Case	Worst Case	Space Complexity
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$

-  **Merge Sort**
 - Divides array evenly; always consistent performance.
 - Requires extra space to merge arrays.
-  **Quick Sort**
 - Very fast in practice due to in-place partitioning.
 - Worst case happens when pivot choices are poor (e.g., already sorted arrays with first/last element as pivot).



Matrix Multiplication Rules

-  **Dimension Rule:**

You can multiply matrix **A** ($m \times n$) with matrix **B** ($n \times p$)

→ The inner dimensions must match: **A's columns = B's rows**

-  **Result Size:**

The resulting matrix will have dimensions ($m \times p$)

-  **Not Commutative:**

In general, $A \times B \neq B \times A$

-  **Associative:**

$$(A \times B) \times C = A \times (B \times C)$$

-  **Distributive:**

$$A \times (B + C) = A \times B + A \times C$$

-  **Multiplying with Zero Matrix:**

Any matrix multiplied with a zero matrix results in a zero matrix

-  **Identity Matrix Rule:**

$A \times I = A$, where I is the identity matrix of appropriate size

Example: Multiply 2×3 and 3×2 Matrices

Let's multiply:

Matrix A (2×3):

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

Matrix B (3×2):

Step 1: Check Dimensions

- A is 2×3 , B is $3 \times 2 \rightarrow$  Multiplication is valid
- Result matrix C will be 2×2

$$B = \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix}$$

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad B = \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix}$$

C[0][0] (1st row of A • 1st column of B):

$$1 \times 7 + 2 \times 9 + 3 \times 11 = 7 + 18 + 33 = \boxed{58}$$

C[0][1] (1st row of A • 2nd column of B):

$$1 \times 8 + 2 \times 10 + 3 \times 12 = 8 + 20 + 36 = \boxed{64}$$

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad B = \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix}$$

C[1][0] (2nd row of A • 1st column of B):

$$4 \times 7 + 5 \times 9 + 6 \times 11 = 28 + 45 + 66 = \boxed{139}$$

C[1][1] (2nd row of A • 2nd column of B):

$$4 \times 8 + 5 \times 10 + 6 \times 12 = 32 + 50 + 72 = \boxed{154}$$

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad B = \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix}$$

✅ Final Result Matrix C (2×2):

$$C = \begin{bmatrix} 58 & 64 \\ 139 & 154 \end{bmatrix}$$

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$C_{11} = A_{11} \cdot B_{11} + A_{12} \cdot B_{21}$$

$$C_{12} = A_{11} \cdot B_{12} + A_{12} \cdot B_{22}$$

$$C_{21} = A_{21} \cdot B_{11} + A_{22} \cdot B_{21}$$

$$C_{22} = A_{21} \cdot B_{12} + A_{22} \cdot B_{22}$$

$$> \boxed{2 + 2 + 2 + 2 = 8 \text{ multiplications}}$$

 **Why Time = $8T(n/2) + \Theta(n^2)$?**

- $8T(n/2)$: 8 recursive multiplications
- $\Theta(n^2)$: cost of **adding submatrices** (each addition is $n/2 \times n/2$, total additions ≈ 4)

symbolic form used in the **Master Theorem**, which is:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + \Theta(n^k \cdot \log^p n)$$

$$T(n) = 8T(n/2) + \Theta(n^2)$$

- $a = 8 \rightarrow$ number of subproblems
- $b = 2 \rightarrow$ input size is halved each time
- $k = 2 \rightarrow$ non-recursive work is $\Theta(n^2)$



Step 3: Apply **Case 1** of Master Theorem

Case 1:

If $f(n) = O(n^{\log_b a - \epsilon})$, then:

$$T(n) = \Theta(n^{\log_b a})$$

So here:

$$T(n) = \Theta(n^{\log_2 8}) = \Theta(n^3)$$



Final Answer:

$$T(n) = \Theta(n^3)$$

Strassen's algorithm



Introduction

- Strassen's algorithm is a faster matrix multiplication method using divide-and-conquer.
- It reduces multiplications from 8 to 7, improving performance over the standard method.
- Time complexity: $\Theta(n^{2.81})$

Strassen's algorithm

Advantages

- Faster than standard $\Theta(n^3)$ for large matrices
- Reduces computational cost
- Useful in high-performance computing and graphics

$$X = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \text{ and } Y = \begin{bmatrix} E & F \\ G & H \end{bmatrix}$$

Formula :

Matrix A: $M_1 := (A + C) \times (E + F)$

| 2 3 | $M_2 := (B + D) \times (G + H)$

| 1 4 | $M_3 := (A - D) \times (E + H)$

Matrix B:

| 1 2 | $M_4 := A \times (F - H)$

| 5 6 | $M_5 := (C + D) \times (E)$

$$M_6 := (A + B) \times (H)$$

$$M_7 := D \times (G - E)$$

$$I := M_2 + M_3 - M_6 - M_7$$

$$J := M_4 + M_6$$

$$K := M_5 + M_7$$

$$L := M_1 - M_3 - M_4 - M_5$$

symbolic form used in the **Master Theorem**, which is:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + \Theta(n^k \cdot \log^p n)$$

$$T(n) = 7 \cdot T\left(\frac{n}{2}\right) + \Theta(n^2)$$

- $a = 7 \rightarrow$ number of subproblems
- $b = 2 \rightarrow$ problem size division
- $k = 2 \rightarrow$ exponent of the non-recursive work
- $p = 0 \rightarrow$ no $\log n$ factor in the non-recursive part

✓ Step 3: Apply Master Theorem Case 1

Case 1 (if $k < \log_b a$):

$$T(n) = \Theta(n^{\log_b a}) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$$

📄 Final Conclusion:

Using $a = 7, b = 2, k = 2, p = 0$:

$$T(n) = 7T(n/2) + \Theta(n^2) \Rightarrow T(n) = \Theta(n^{2.81})$$

MatrixMultiply(A, B, n):

Input: Two $n \times n$ matrices A and B

Output: Product matrix $C = A \times B$

if $n == 1$: return $[[A[0][0] * B[0][0]]]$

Partition A into A11, A12, A21, A22 (each $n/2 \times n/2$)

Partition B into B11, B12, B21, B22

Compute the following recursively:

M1 = MatrixMultiply(A11, B11)

M2 = MatrixMultiply(A12, B21)

M3 = MatrixMultiply(A11, B12)

M4 = MatrixMultiply(A12, B22)

M5 = MatrixMultiply(A21, B11)

M6 = MatrixMultiply(A22, B21)

M7 = MatrixMultiply(A21, B12)

M8 = MatrixMultiply(A22, B22)

Combine to form the result submatrices:

C11 = M1 + M2

C12 = M3 + M4

C21 = M5 + M6

C22 = M7 + M8

Merge C11, C12, C21, C22 into single matrix C return C

Algorithm

Summary

Size	Method	Use Formula	Time Complexity
Any $n \times n$	Normal	$\Theta(n^3)$	e.g., 1024^3
Any $n \times n$	Strassen	$\Theta(n^{\log_2 7})$	e.g., $1024^{2.81}$



Max-Min Problem

1. Simple Linear Approach

- **Algorithm:**
 - Traverse the array once.
 - Compare each element with current min and max.
- **Comparisons:**
 - For each element (after the first), 2 comparisons are made (one with min, one with max).
 - Total comparisons: $2(n - 1)$.
- **Time Complexity:**
 $O(n)$.

✓ Example

Let's say `arr = [3, 5, 1, 2, 4, 8]`

1. Split into two: `[3, 5, 1]` and `[2, 4, 8]`

2. Recurse:

- `[3, 5, 1] → [3]` and `[5, 1]`
- `[5, 1] → min=1, max=5`
- Combine with `[3]`: `min = 1, max = 5`
- Similarly process `[2, 4, 8] → min = 2, max = 8`

3. Final step: combine (1, 5) and (2, 8):

- `min = min(1, 2) = 1`
- `max = max(5, 8) = 8`

✓ Final result: `min = 1, max = 8`

✓ Time Complexity

$$T(n) = 2T(n/2) + 2$$

1. if $a > b^k$, then $T(n) = \theta(n^{\log_b a})$
2. if $a = b^k$, then
 - (a) if $p > -1$, then $T(n) = \theta(n^{\log_b a} \log^{p+1} n)$
 - (b) if $p = -1$, then $T(n) = \theta(n^{\log_b a} \log \log n)$
 - (c) if $p < -1$, then $T(n) = \theta(n^{\log_b a})$

master theorem

3. if $a < b^k$, then
 - (a) if $p \geq 0$, then $T(n) = \theta(n^k \log^p n)$
 - (b) if $p < 0$, then $T(n) = \theta(n^k)$

✓ Master Theorem - Case 1:

If $f(n) = O(n^d)$ where $d < \log_b a$, then:

$$T(n) = \Theta(n^{\log_b a}) = \Theta(n)$$

✓ Final Result:

- Time Complexity: $T(n) = \Theta(n)$

Step 1: Expand the recurrence

$$\begin{aligned}T(n) &= 2T\left(\frac{n}{2}\right) + 2 \\&= 2\left[2T\left(\frac{n}{4}\right) + 2\right] + 2 = 4T\left(\frac{n}{4}\right) + 4 + 2 \\&= 8T\left(\frac{n}{8}\right) + 8 + 4 + 2\end{aligned}$$

...

After $\log_2 n$ steps:

$$T(n) = 2^k T(1) + 2(2^{k-1} + 2^{k-2} + \dots + 1)$$

That sum is a geometric series:

$$= 2^k T(1) + 2(2^k - 1)$$

$$T(n) = \frac{3n}{2} - 2$$

n	Linear Comparisons	Divide & Conquer
–	$2(n - 1)$	$\frac{3n}{2} - 2$
2	2	1
4	6	5
8	14	10
16	30	22
32	62	46
64	126	94

FUNCTION FindMinMax(arr, low, high, minVal, maxVal):

IF low == high THEN

// Only one element

minVal ← arr[low]

maxVal ← arr[low]

RETURN

ELSE

mid ← (low + high) / 2

DECLARE min1, max1, min2, max2

CALL FindMinMax(arr, low, mid, min1, max1)

CALL FindMinMax(arr, mid + 1, high, min2, max2)

minVal ← MIN(min1, min2)

maxVal ← MAX(max1, max2)

RETURN

ELSE IF high == low + 1 THEN

// Two elements

IF arr[low] < arr[high] THEN

minVal ← arr[low]

maxVal ← arr[high]

ELSE

minVal ← arr[high]

maxVal ← arr[low]

ENDIF

RETURN